

CHAPTER 1

OVERVIEW OF C

Key Points

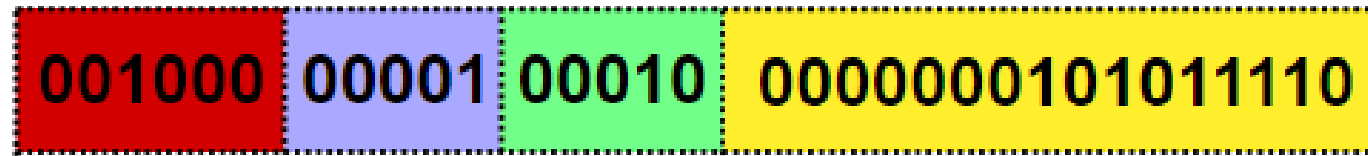
- Why Programming?
- Brief History of C
- Importance of C
- Sample Programs
- Basic Structure of C Program
- Executing a C Program

Why Programming?



Why Programming?

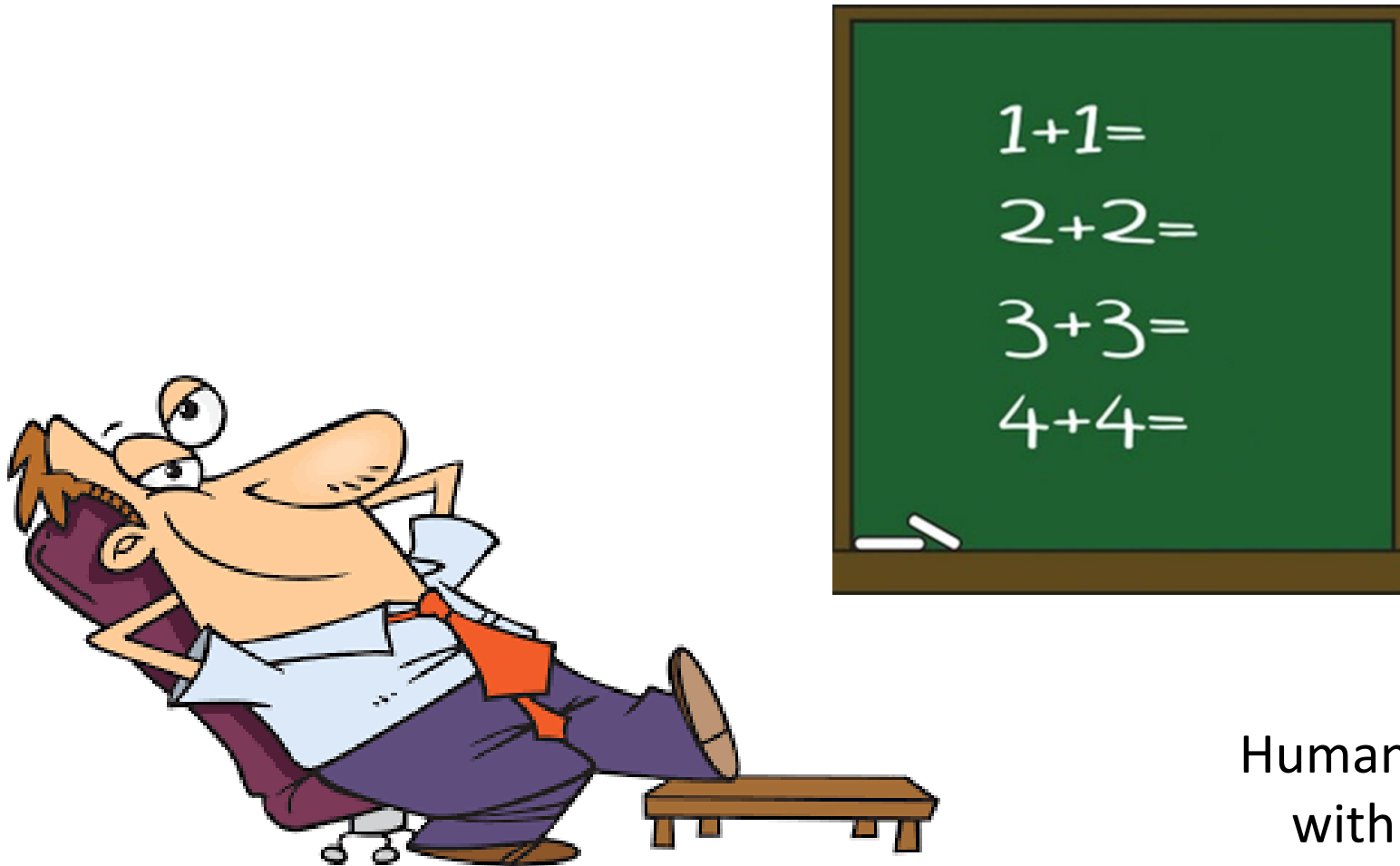
Higher Level Language **Vs.** Machine Language



Is a 32 bit instruction in a binary understandable by any one?

Why Programming?

Higher Level Language **Vs.** Machine Language

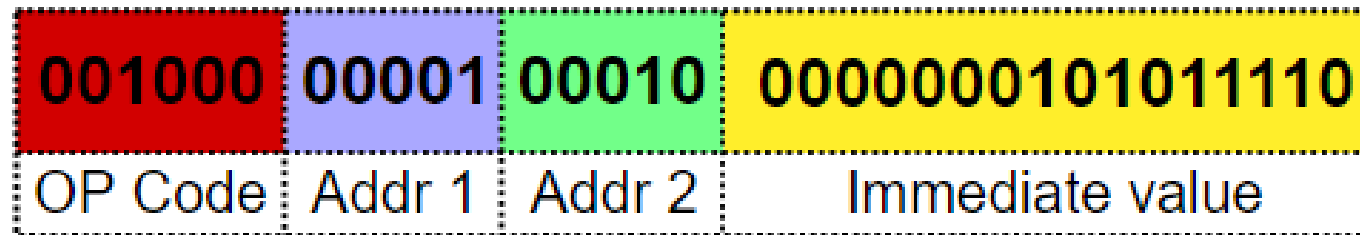


Humans are more comfortable
with higher level language

Why Programming?

Higher Level Language **Vs.** Machine Language

MIPS32 Add Immediate Instruction



Equivalent mnemonic: **addi \$r1, \$r2, 350**

The equivalent mnemonic of a binary instruction can be understandable but still difficult. (what is \$r1, \$r2 ??)

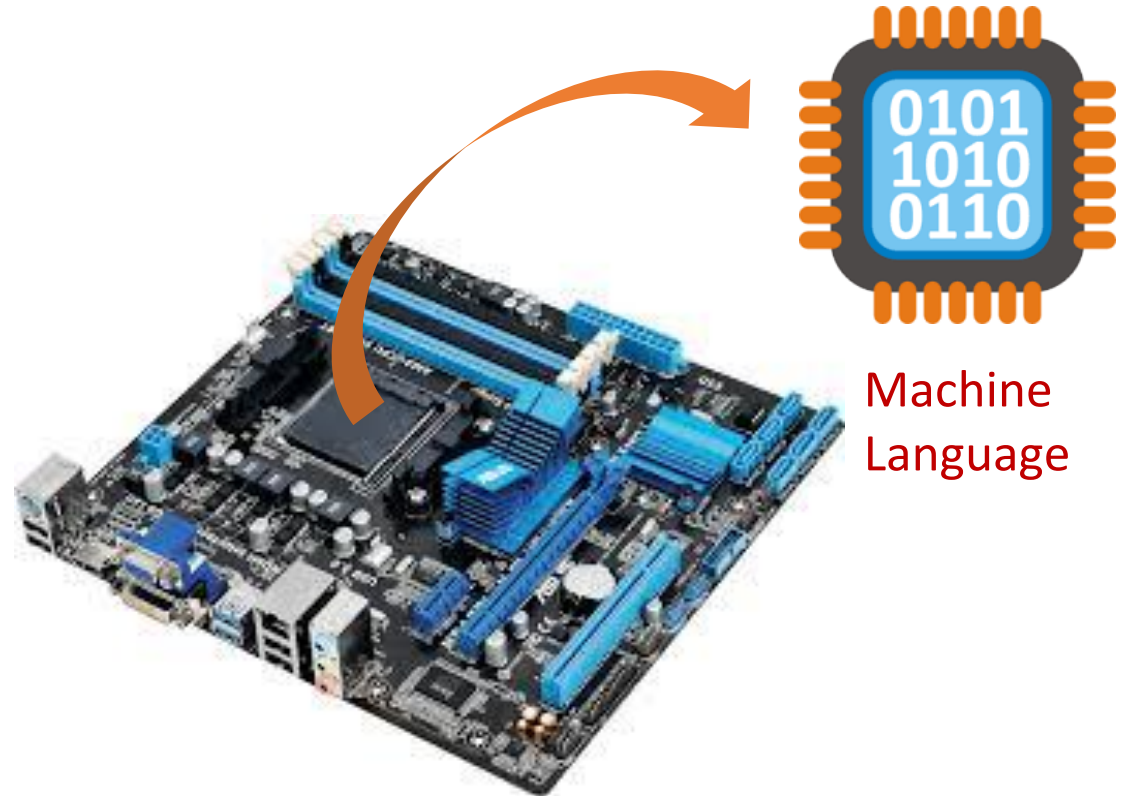
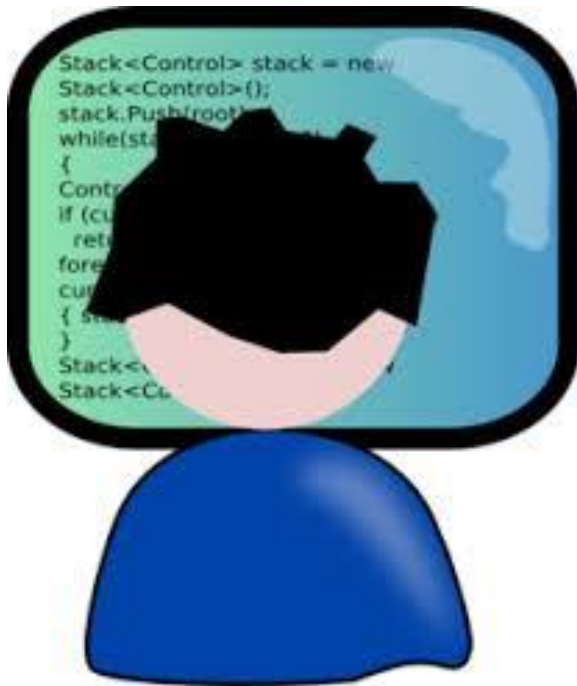
Instead, $a = 5$, $b = 6$ and $c = a + b$

Can be understood by all

Why Programming?

Higher Level Language Vs. Machine Language

Higher Level Language
C language

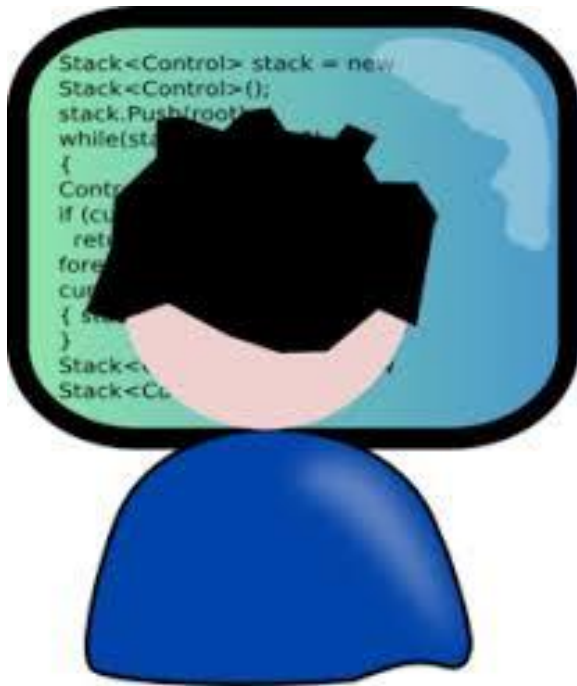


Machine
Language

Write a program in higher level language and convert it into machine language

Higher Level Language Vs. Machine Language

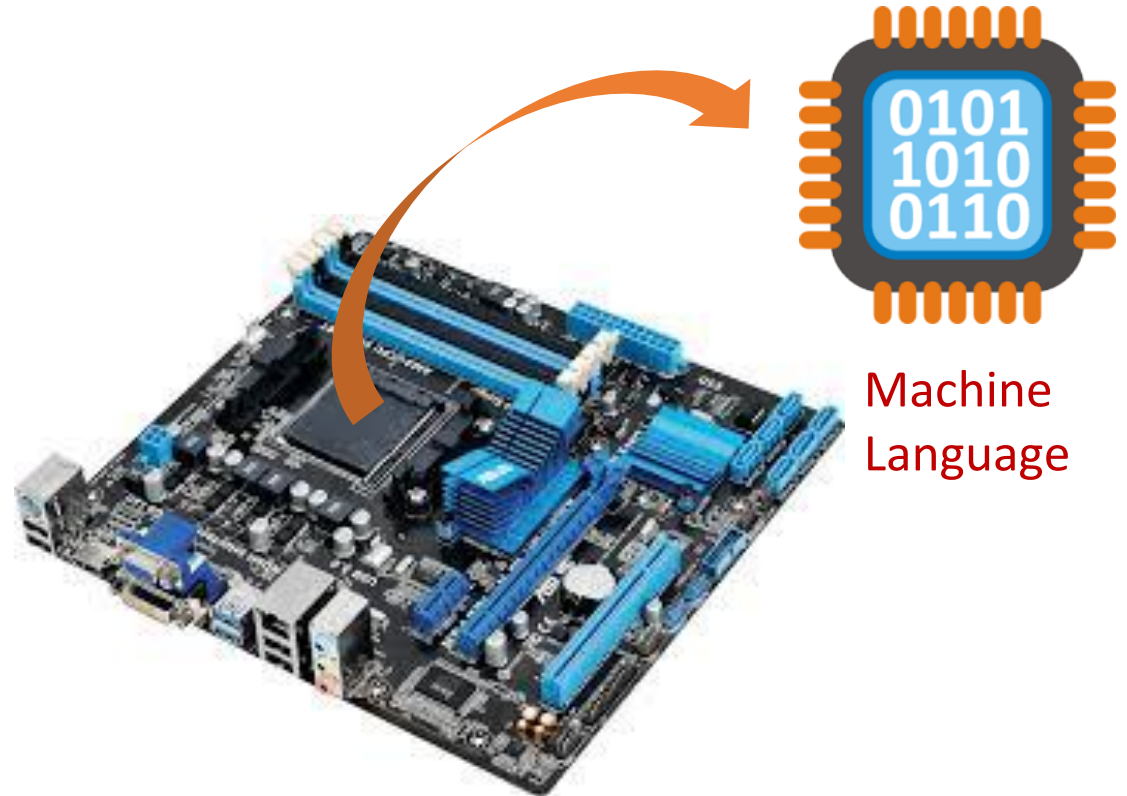
Higher Level Language
C language



- Compiler
- Assembler



- Interpreter



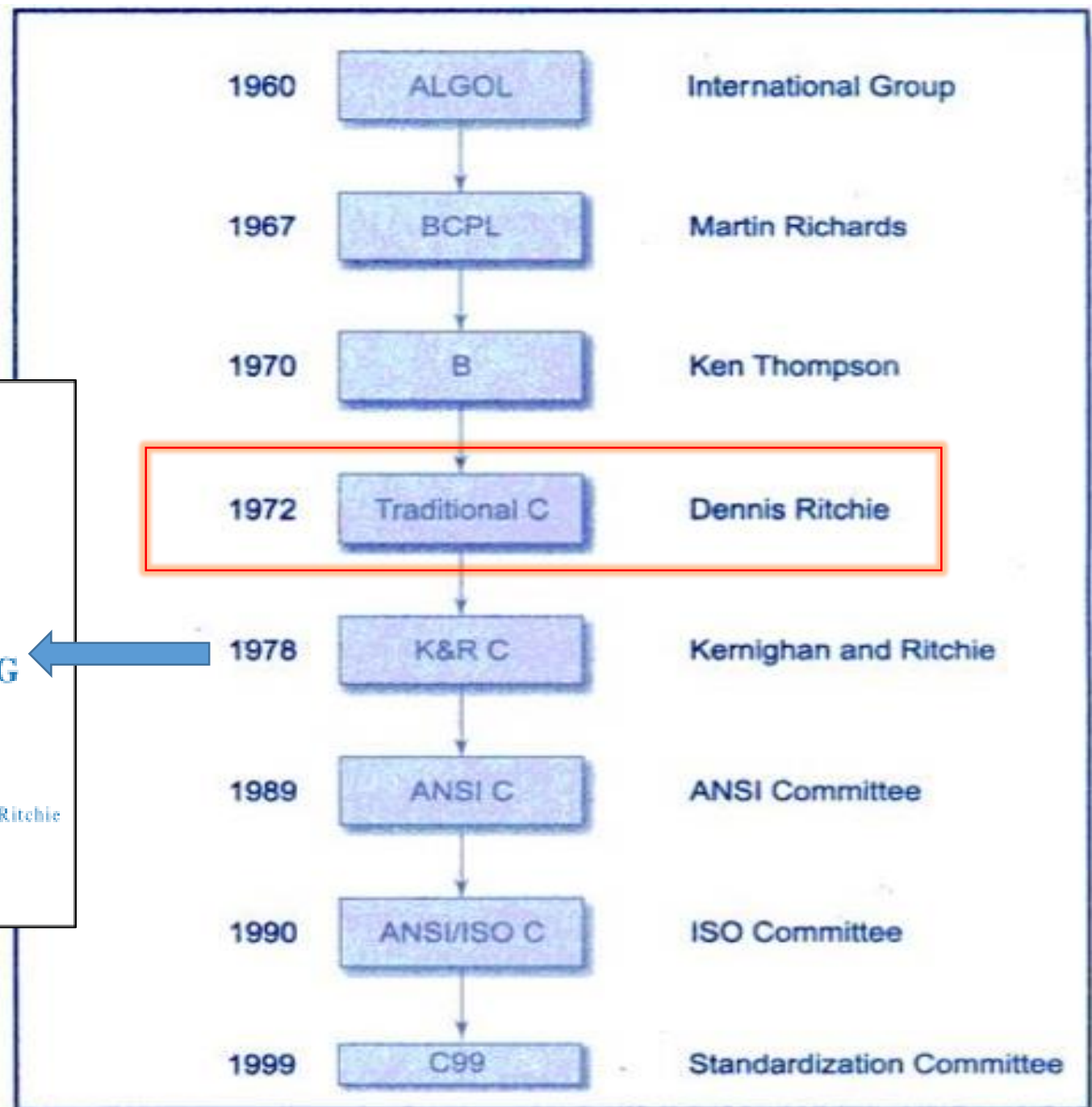
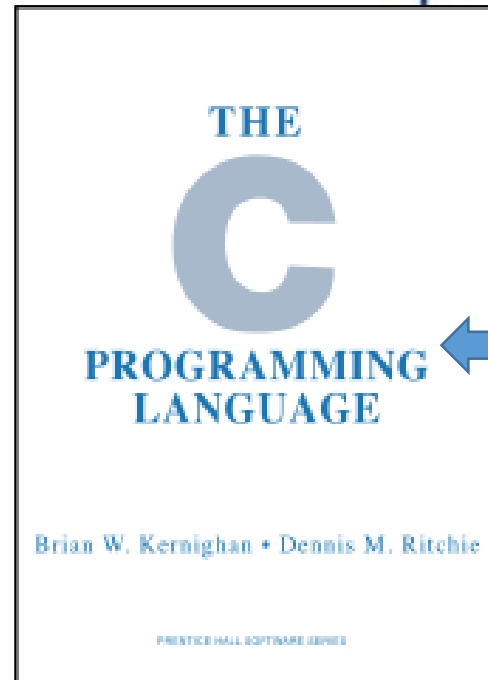
Machine
Language

Compiler, Assembler and Interpreter will convert higher level language program into a machine language.

Brief History of 'C'



Dennis Ritchie
Bell labs, USA



Overview of C : Importance of C

- High level language
- Rich set of built-in functions and operators
- Only 32 keywords in ANSI C
- Highly portable
- Suited for structured programming (Modules / Blocks)
- Modular programming: debugging, testing and maintenance easy
- Ability to extend: can add user define functions to C library

Overview of C: Sample Program

```
/* 1: Printing a Message */
main()
{
    /* --- Printing Begins --- */
    printf("I See, I Remember!");
    /* --- Printing Ends --- */
}
```

```
/* 2: Add Values */
#include<stdio.h>
main()
{
    int number;
    float amount;
    number=100;
    amount = 25.25+74.75;
    printf("%d\n", number);
    printf("%f\n", amount);
}
```

```
/* 3: Trigonometry */
#include<stdio.h>
#include<math.h>
#define PI 3.14
main()
{
    double angle, rad;
    angle = 90;
    rad = (angle/180)*PI;
    printf("sin(%f)=%f\n", rad, sin(rad));
    printf("cos(%f)=%f\n", rad, cos(rad));
}
```

Overview of C: Sample Program

- **main()** Special function
- Where program starts
- **MUST** have **exactly ONE main()** function otherwise ...
- Compiler: Which one marks beginning of the program?

1 sample code.c

```
main()  
{  
/* --- Printing Begins --- */  
printf("I See, I Remember!");  
/* --- Printing Ends --- */  
}
```

Overview of C: Sample Program

1_sample_code.c

```
main()
```

```
{
```

```
/* --- Printing Begins --- */
```

```
printf("I See, I Remember!");
```

```
/* --- Printing Ends --- */
```

```
}
```

Function
body

Set of
Instructions

“ { ” Beginning &
“ } ” End of
a `main()` function.

Overview of C: Sample Program

- Executable statement
- Predefine standard C function
- Every statement in C should end with semicolon (;)

1_sample_code.c

```
main()  
{  
/* --- Printing Begins --- */  
printf("I See, I Remember!");  
/* --- Printing Ends --- */  
}
```

Overview of C: Sample Program

1_sample_code.c

```
main()  
{  
/* --- Printing Begins --- */  
printf("I See, I Remember!");  
/* --- Printing Ends --- */  
}
```



- Comment Lines
 - **Non executable** statements
 - Ignored by compiler
 - Can not be nested
- **DO NOT** effect execution **speed** and **size** of compiled program

Overview of C: Sample Program

- `main()` Special function
- Where program starts
- **MUST** have **exactly ONE** `main()` function otherwise ...
- Compiler: Which one marks beginning of the program?

- Executable statement
- Predefine standard C function
- Every statement in C should end with semicolon (;)

`1_sample_code.c`

```
main()  
{  
    /* --- Printing Begins --- */  
    printf("I See, I Remember!");  
    /* --- Printing Ends --- */  
}
```

Function
body

Set of
Instructions

“ { ” Beginning &
“ } ” End of
a `main()` function.

- Comment Lines
- **Non executable** statements
- Ignored by compiler
- Can not be nested

- **DO NOT** effect execution **speed** and **size** of compiled program

Overview of C: Sample Program

- Program that uses I/O library functions
- Not necessary for *printf* and *scanf*

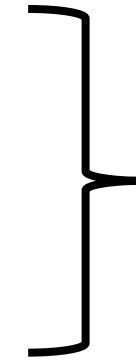
```
/* 1: Printing a Message */
#include<stdio.h>
main()
{
    /* --- Printing Begins --- */
    printf("I See, I Remember!");
    /* --- Printing Ends --- */
}
```

- Distinct between **UPPER CASE** and **lower case**
- **printf** and **PRINTF** are not same
- **Upper case** used for **symbolic constants**

Overview of C : The main() Function

void before main() :
Function **DO NOT**
return any
information to OS

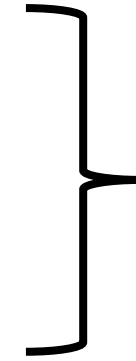
- main ()
- int main ()
- void main ()



Empty
parentheses:
function has no
arguments

int before main() :
Function **DO**
return integer
value to OS

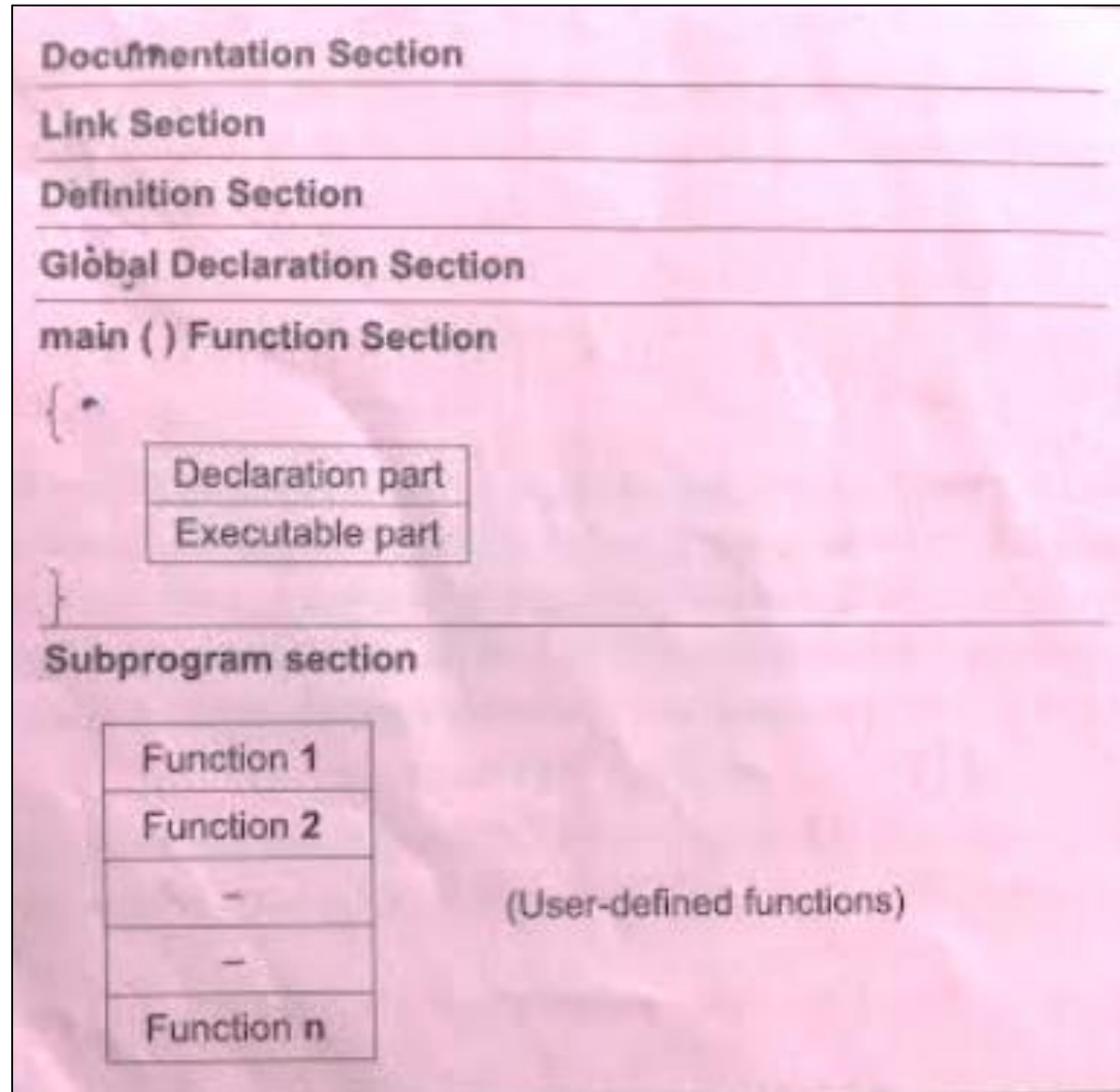
- main (void)
- int main (void)
- void main (void)



Indicate No
arguments by
using **void**
keyword

```
int main() or int main(void)
{
return 0; /* last statement */
}
```

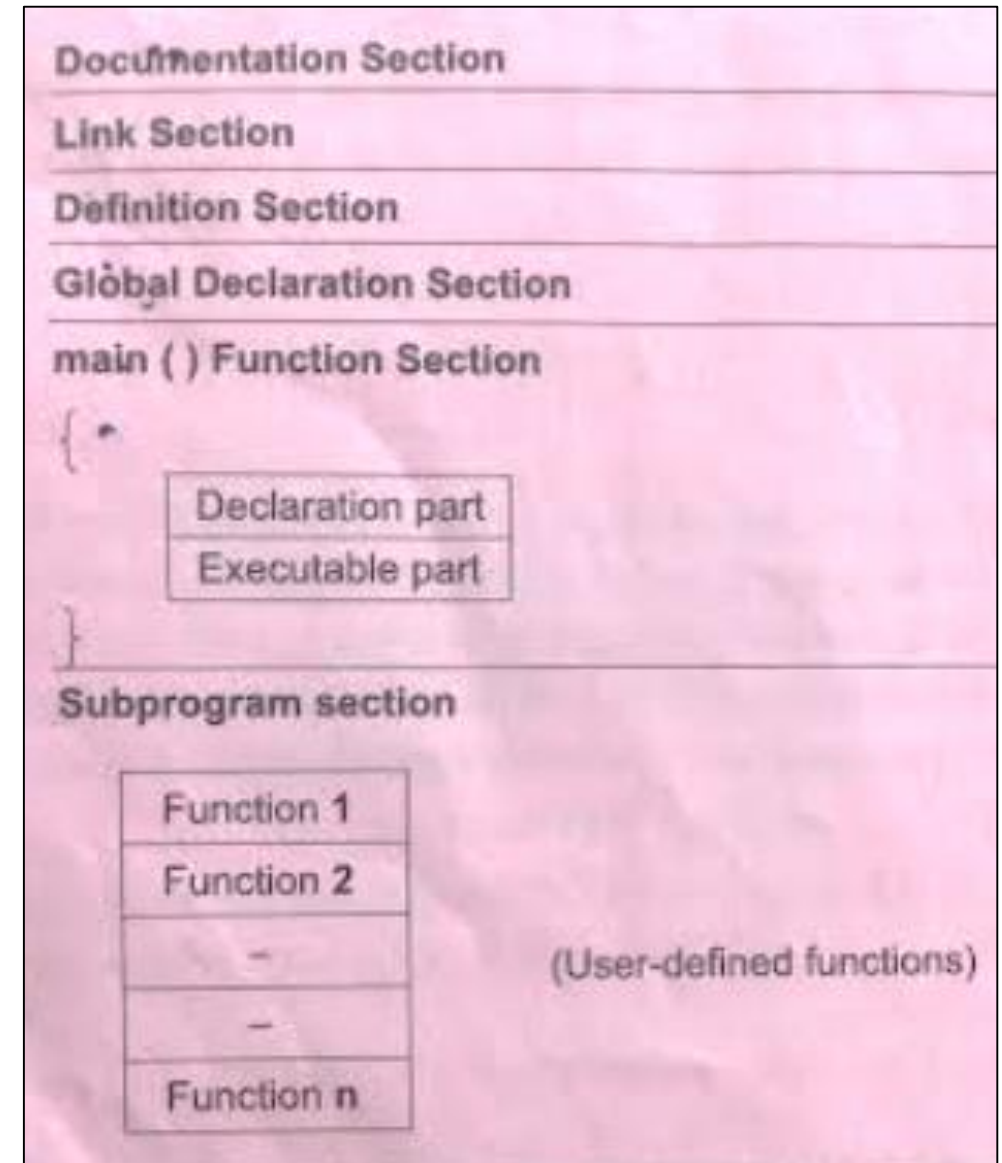
Overview of C : Basic Structure of C Program



Overview of C : Basic Structure of C Program

add.c

```
1 /* Prog. to add two numbers
2   Author: Raam */ /*..... Documentation*/
3
4 #include<stdio.h> /*..... Linking */
5
6 #define PI 3.14 /*..... Definition */
7
8 int g; /*..... Global declaration */
9
10 int main() /*..... main() function */
11 {
12     int a,b,sum; /* Declaration part */
13     a=4; b=5; /* Executable part*/
14     sum = a+b;
15     printf("Addition = %d\n",sum);
16     return 0;
17 }
18 /* User define function starts from here */
```



Overview of C : Basic Structure of C Program

Program
Execution begins
at opening
bracket

Closing bracket is
a logical end of
the program

```
1 /* Prog. to add two numbers
2   Author: Raam */ /*..... Documentation*/
3
4 #include<stdio.h> /*..... Linking: tell compiler to link function
5                               from library */
6 #define PI 3.14 /*..... Definition: symbolic constants */
7
8 int g; /*..... Global declaration: global variables used in
9                               more than one function */
10
11 int main() /*..... main() function */
12 {
13     int a,b,sum; /* Declaration part: Daclare all variable used
14                               in executable part */
15     a=4; b=5; /* Executable part*/
16     sum = a+b;
17     printf("Addition = %d\n",sum);
18     return 0;
19 }
20 /* User define function starts from here */
```

NOTE: All section **except main()** function section may be absent when they are not required.

Overview of C : Executing a 'C' Program

1. Creating the program
2. Compiling the program
3. Linking the program with functions needed from C library
4. Executing the program
(Handled by OS)

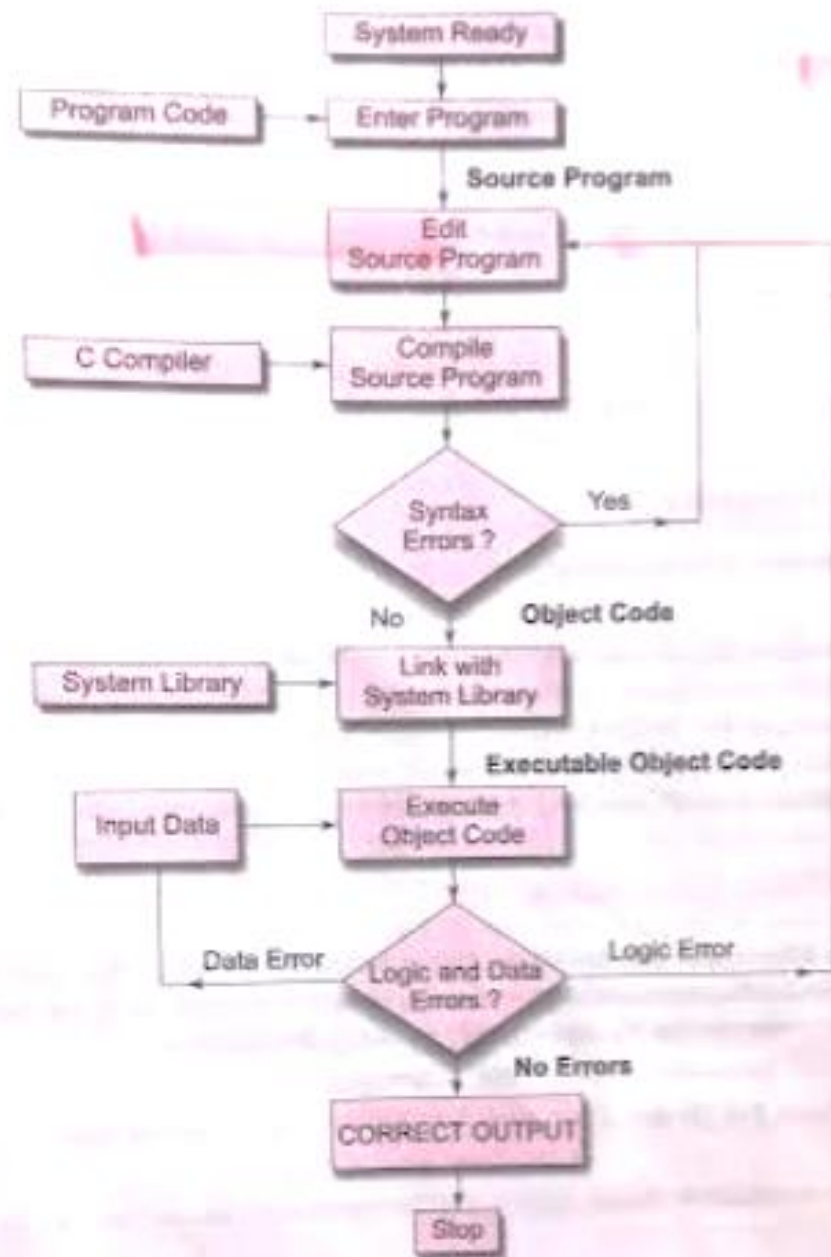


Fig. 1.10 Process of compiling and running a C program

Overview of C : Executing a 'C' Program: Linux OS

1. Creating the program
2. Compiling the program
3. Linking the program with functions needed from C library
4. Executing the program
(Handled by OS)

1. `$ gedit add.c` →

2,3. `$ gcc add.c`

4. `$ ./a.out`

```
1 /* Prog. to add two numbers
2   Author: Raam */ /*..... Documentation*/
3
4 #include<stdio.h> /*..... Linking */
5
6 #define PI 3.14 /*..... Definition */
7
8 int g; /*..... Global declaration */
9
10 int main() /*..... main() function */
11 {
12     int a,b,sum; /* Declaration part */
13     a=4; b=5; /* Executable part*/
14     sum = a+b;
15     printf("Addition = %d\n",sum);
16     return 0;
17 }
18 /* User define function starts from here */
```

```
$ gcc -o my_add add.c
$ ./my_add
```

Output: `Addition = 9`

Overview of C : Compilation Steps

Source code: `program_name.c`

1. Pre-processing

- Comments are removed
- Expansion of Included files and Macros `program_name.i`
- Conditional compilation

2. Compilation

- Converts preprocessed code to assembly language `program_name.s`

3. Assembly

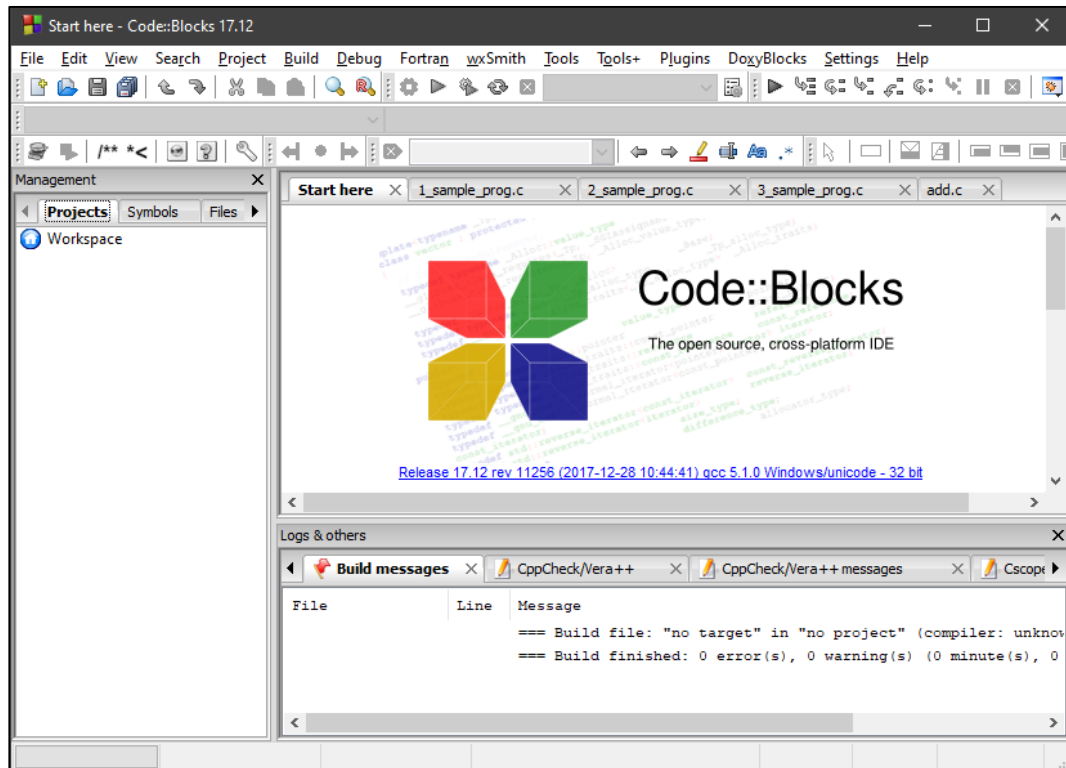
- Converts assembly language code to machine language → generate object file `program_name.o`

4. Linking

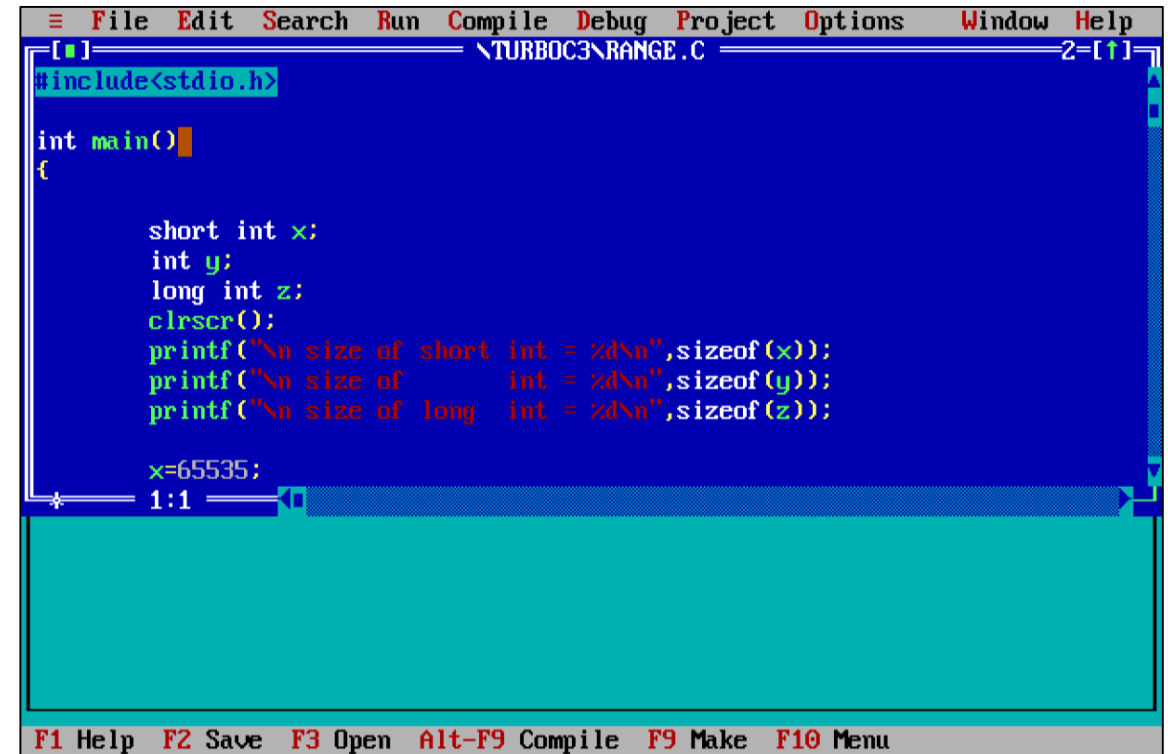
- Links object files to generate executable file `program_name / a.out / program_name.exe`

Overview of C : Executing a 'C' Program: Windows OS

1. Code Blocks IDE



2. Turbo C



Overview of C : Executing a 'C' Program: Windows OS

1. Code Blocks IDE

1. Creating the program
2. Compiling the program
3. Linking the program with functions needed from C library
4. Executing the program
(Handled by OS)

1. Create in IDE & Save

2,3. Build / Ctrl+F9

4. Run / Ctrl+F10

Output file: program_name.exe

Overview of C : Executing a 'C' Program: Windows OS

2. Turbo C

1. Creating the program
2. Compiling the program
3. Linking the program with functions needed from C library
4. Executing the program
(Handled by OS)

1. Create & Save

2,3. Compile / Alt+F9

4. Run / Ctrl+F9

Output file: program_name.exe